Software Requirements Specification

FetchKart - E-Commerce Platform

Author: Rudra Sagar Gondhalekar

Document Version: 1.0

1. Introduction

1.1 Purpose

The purpose of this document is to outline the software requirements for **FetchKart**, a scalable and user-centric e-commerce web platform. It defines the system's architecture, functional capabilities, and technical constraints to provide a seamless online shopping experience.

1.2 Project Scope

FetchKart is a foundational e-commerce application built with a modern Glassmorphism UI. It handles the core lifecycle of an online store: user authentication, product browsing, dynamic cart management, and simulated checkout/payment flows. It is designed to run on lightweight local or cloud environments and caters to both standard shoppers and sellers.

1.3 Technology Stack

- **Frontend:** HTML5, CSS3 (Glassmorphism design language), JavaScript.
- **Backend:** PHP (7.x / 8.x) for server-side processing, APIs, and session management.
- **Database:** MySQL.
- **Environment:** Apache Server (XAMPP, WAMP, or similar).

2. Overall Description

2.1 Product Perspective

FetchKart follows a standard client-server web architecture. The front end provides an interactive user interface that communicates with the PHP backend. PHP handles

application logic, cart state tracking via sessions, and reads/writes to the MySQL database.

2.2 User Classes and Characteristics

- **Customers:** General users who visit the site to browse products, add items to their shopping cart, apply discount codes, and complete simulated purchases.
- **Sellers:** Vendor users who use the platform (`seller.html`) to manage inventory and view platform sales.

2.3 Operating Environment

- **Server-Side:** Any environment supporting Apache and PHP 7+.
- **Client-Side:** Modern web browsers (Chrome, Edge, Firefox, Safari) on desktop or mobile devices.

3. System Features & Functional Requirements

3.1 User Authentication

- **FR 1.1:** The system shall allow new users to register via the `signup.html` page.
- **FR 1.2:** The system shall authenticate returning users via the `login.html` interface.
- **FR 1.3:** The backend shall manage user sessions securely using PHP.

3.2 Product Catalog

- **FR 2.1:** The system shall display available items in a clean browse layout on the `shop.html` and `index.html` pages.
- **FR 2.2:** Users shall be able to view detailed information about individual items (`product.html`).
- **FR 2.3:** Sellers shall be able to interact with the catalog through a dedicated seller dashboard (`seller.html`).

3.3 Cart Management

- **FR 3.1:** Users shall be able to add products to a shopping cart (`cart.html`).
- **FR 3.2:** The cart state must be tracked dynamically across the site using PHP session variables.
- **FR 3.3:** The cart interface must allow users to view their total price, update item quantities, and remove unwanted items.

3.4 Checkout and Payments

- **FR 4.1:** The system shall provide a simulated checkout flow (`checkout.html`) to collect shipping and billing information.
- **FR 4.2:** Users shall be able to enter promotional codes, which the system will validate using `check_coupons.php`.
- **FR 4.3:** The platform shall simulate a modern Indian payment gateway flow, specifically supporting UPI Payments (`upi_payment.html`).
- **FR 4.4:** Upon successful checkout, the system shall log the transaction and make it viewable on the user's `orders.html` page.

4. Non-Functional Requirements

4.1 UI / UX Requirements

Design Language: The application must utilize a "Glassmorphism" design approach, characterized by translucent, frosted-glass-like UI components to ensure a modern visual appeal.

Responsiveness: The layout must adapt gracefully to different screen sizes.

4.2 Performance

Because the cart relies on PHP session-based state tracking rather than continuous database querying, cart updates (adding/removing items) must reflect instantly without heavy database load.

4.3 Deployment & Setup Requirements

The application must be deployable simply by cloning the repository into an `htdocs` (or equivalent web root) directory. Database initialization must be streamlined using the provided `migrate.php` or by importing `database.sql`.

5. System Interfaces

5.1 Application Programming Interfaces (APIs)

The system uses an internal API structure (housed in the `/api/` directory) to separate backend logic from frontend presentation, allowing asynchronous data fetching (via JavaScript) for smoother user interactions.

5.2 Database Interface

The application interfaces with a relational MySQL database. Based on the system's features, the expected schema relies on tables for `Users`, `Products`, `Orders`, and `Coupons`.